

5. 步骤四：TreeBuilder 解析最终 URI

Parser 会在临时 VikingFS 下创建一个文档根目录。TreeBuilder.finalize_from_temp 读取这个临时目录，要求其中只有一个有效文档根目录，然后计算正式 root_uri。这个阶段只确定结构和 URI，不直接生成摘要。

```
TreeBuilder.finalize_from_temp(temp_dir_path, ctx, scope, to_uri, parent_uri)
-> ls(temp_uri)，找到唯一文档根目录
-> sanitize document name
-> 如果 source_format == repository，Git URL 可转为 org/repo
-> base_uri = parent_uri 或 viking://resources
-> candidate_uri = to_uri 或 base_uri / final_doc_name
-> 若未指定 to_uri，则自动 resolve_unique_uri
-> 返回 BuildingTree(root.uri=root_uri, root.temp_uri=temp_doc_uri)
```

场景	root_uri 规则
指定 to	使用 to 作为精确目标 URI，要求资源接口只写 resources scope。
指定 parent	检查 parent 存在且为目录，在 parent 下拼接文档名。
未指定 to/parent	默认落到 viking://resources/{文档名}。
名称冲突	自动追加 _1、_2 ... 直到找到可用 URI。
代码仓库	若可解析 org/repo，则最终路径用仓库组织名和仓库名表达。
媒体资源	根据媒体类型选择相应资源基路径。

6. 步骤五：正式落盘、生命周期锁与增量更新

TreeBuilder 返回 root_uri 与 temp_uri 后，ResourceProcessor 负责把临时文档树移动到正式 AGFS。这里分首次添加和增量更新两种路径。

路径	判断条件	处理方式
首次添加	root_uri 不存在	创建父目录，在 point lock 保护下把 temp_uri 对应目录 mv 到 root_uri。
增量更新	root_uri 已存在	不立即覆盖目标目录，而是为目标目录获取 SUBTREE lifecycle lock，后续由 SemanticDag 做 diff 同步。
锁失败	无法获取 lifecycle lock	优雅降级为空 handle，但会记录 warning；后续处理仍尽量继续。
finalize 失败	TreeBuilder 或移动前出错	删除临时目录，返回 error，避免污染正式知识库。

为什么要先写临时目录

临时目录让解析阶段和正式知识库解耦：解析失败不会留下半成品；首次添加可快速移动落盘；增量更新可先